

Noah Le

7/25/26

CS – 532

Unan

Bonus HW Report

1. Summary

For this project, I wrote a C program to handle a producer-consumer problem using pthreads and pipes for the IPC.

Producers: The parent process creates 10 producer threads. Trying to guarantee unique random numbers was a little challenging, so I ended up just making an array from 0 to 1000, shuffling it, and grabbing the first 500 values to push to the pipe. I used a `pthread_mutex_t` lock on the write system call, so the threads didn't step on each other's toes while writing.

Signals (CS532 Req): To complete the graduate requirement, I went with a `SIGUSR1` signal. The child process calls `pause()` and basically hangs there. When the producers finish up, the parent closes the write end of the pipe and sends the signal. The child catches it, flips a `sig_atomic_t` flag, and knows it's clear to start reading.

Consumers: After the signal is received, the child process starts 20 consumer threads. To prevent race conditions during the pipe reads, I used a POSIX semaphore to ensure only one consumer accesses the pipe at a time. Every thread reads 250 numbers from the pipe and keeps a local running total. The main thread just waits on all of them to finish, adds their totals together to get the average, and prints it.

Bonus: For the extra points, I added a quick progress tracker so you can see how many writes the producers have done. I sent those updates to `stderr` instead of regular output. Because of that, the normal standard output stays clear, so when you run `> output.txt`, it only grabs the actual final average like it's supposed to.

2. How to Run

Quick Note: I routed all the progress tracking logs to stderr. That way, stdout stays completely clean, and output redirection works without dumping all the progress bars into the text file.

Compiling

Just make sure the Makefile and bonusHW.c are in the same folder, then run:

```
$ make
```

Run the program and redirect the output like this. You'll see the progress tracker running in your terminal, but only the final computed average gets sent to the text file:

```
$ ./bonusHW > output.txt
```

To see the final average, just read the file:

```
$ cat output.txt
```

3. AI Tool Documentation

For this project, I used Copilot to aid me in this assignment, and it was a huge help for figuring out synchronization and catching bugs. I wasn't sure how to use a mutex or a semaphore to keep my 10 producer threads from overwriting each other in the shared pipe. Copilot explained that a semaphore is basically just a counter and a mutex acts as a strict ownership lock, which is what I needed for a critical section like a pipe write. It also helped me map out the CS 532 graduate requirement. I wanted the child process to halt using `pause()` until the parent was done, but I wasn't totally sure how to do that. The tool showed me how to set up a volatile `sig_atomic_t` flag that gets flipped by a signal handler catching `SIGUSR1`. Copilot pointed out that `pause()` can wake up from random background signals, so I had to wrap the `pause()` inside a `while(!flag)` loop to force the child back to sleep if it wasn't the actual wake-up call. Finally, I asked Copilot if it was fine to just pass my loop counter's address (`&i`) directly into `pthread_create` for my thread IDs. It immediately warned me that this causes massive race conditions since the loop keeps spinning and changing `i`, meaning threads would grab duplicate or garbage IDs. To fix this, I went with its suggestion of using dynamic memory allocation. I used `malloc(sizeof(int))` to allocate unique memory for each thread's ID before creating it, and then called `free()` inside the actual producer and consumer thread functions to release the memory once the ID was safely

extracted. Ultimately, based on the Copilot's explanation of binary semaphores versus mutexes, I used a `pthread_mutex_t` to ensure strict ownership for the producer writes, and utilized a `sem_t` binary semaphore for the consumer reads.

Specific Prompt and Output:

‘explain the difference between a semaphore and a mutex in C? I have multiple producer threads that need to write to the same pipe, and I want to make sure i use both of them properly in my c program’

OUTPUT: **Short answer:** Use a **mutex** to protect your pipe writes. Use **semaphores** only when you need counting or coordination (e.g., producer/consumer queues). For your case—multiple producers writing to *one* pipe—a **mutex is the correct tool**.

Below is a clean, exam-ready explanation that still gives you the practical intuition you need for your C program.

Mutex vs Semaphore in C (Clear, Practical, Exam-Ready)

1. What a mutex is

A **mutex (mutual exclusion lock)** is a binary lock that ensures **only one thread** can enter a critical section at a time.

- Only one thread can lock it.
- If another thread tries to lock it, it blocks until the mutex is unlocked.
- Ownership matters: the thread that locks must unlock.

Use case: Protecting shared resources like a pipe, file descriptor, shared buffer, or global variable.

2. What a semaphore is

A **semaphore** is a counter with atomic operations `sem_wait()` and `sem_post()`.

Two types:

- **Binary semaphore** (value 0 or 1): behaves *similar* to a mutex but has no ownership rules.
- **Counting semaphore:** allows multiple threads to proceed up to a limit (e.g., 5 threads allowed in at once).

Use case: Coordinating availability of resources, producer/consumer queues, limiting concurrency.

✂ Key Differences (Simple and Precise)

Ownership

- **Mutex:** has an owner; only the locking thread may unlock.
- **Semaphore:** no ownership; any thread may `sem_post()`.

Purpose

- **Mutex:** mutual exclusion.
- **Semaphore:** signaling or counting.

Behavior

- **Mutex:** always binary.
- **Semaphore:** can be binary or counting.

Correct tool for pipe writes

You want **exactly one producer thread writing at a time** → **mutex**.

A semaphore would allow multiple threads to enter if its count > 1, which is unsafe for pipe writes.

🔧 How to use them properly in your C program

Protecting pipe writes (use a mutex)

c

```
pthread_mutex_t pipe_lock = PTHREAD_MUTEX_INITIALIZER;
```

```
void *producer(void *arg) {
```

```
    pthread_mutex_lock(&pipe_lock);
```

```
    write(pipe_fd, buffer, len); // safe: only one thread writes at a time
```

```
pthread_mutex_unlock(&pipe_lock);  
return NULL;  
}
```

If you also need producer/consumer coordination (use a semaphore)

Example: producers fill a queue, consumer reads from it.

c

```
sem_t items;
```

```
sem_init(&items, 0, 0); // start with 0 items
```

```
void *producer(void *arg) {  
    // produce item...  
    enqueue(item);  
    sem_post(&items); // signal consumer  
}
```

```
void *consumer(void *arg) {  
    sem_wait(&items); // wait until an item exists  
    item = dequeue();  
}
```

Bottom line for your program

- **Use a mutex** to protect the pipe write section.
- **Use semaphores** only if you need to coordinate producer/consumer behavior or limit concurrency.

If you want, I can help you write a full thread-safe producer pipeline with both mutexes and semaphores wired correctly.

‘how do I make a child process completely pause its execution until it receives a signal from the parent process in C? I need the parent to finish all its threads before the child starts reading. can we use a pause() approach’

OUTPUT: 🛠️ **How the pause-based approach works**

1. You define a global flag

This flag tells the child when it’s allowed to start reading:

c

```
volatile sig_atomic_t start_reading = 0;
```

sig_atomic_t ensures the flag can be safely modified inside a signal handler.

2. You install a SIGUSR1 handler using sigaction

The handler’s only job is to flip the flag:

c

```
void start_handler(int sig) {  
    start_reading = 1;  
}
```

```
struct sigaction sa;
```

```
sa.sa_handler = start_handler;
```

```
sigemptyset(&sa.sa_mask);
```

```
sa.sa_flags = 0;
```

```
sigaction(SIGUSR1, &sa, NULL);
```

This ensures that when the parent sends SIGUSR1, the child’s flag becomes 1.

3. The child blocks itself using a while (!start_reading) pause(); loop

This is the key part:

```
c
while (!start_reading) {
    pause(); // Child sleeps until *any* signal arrives
}
```

Here's what happens:

- `pause()` suspends the child.
- When the parent sends `SIGUSR1`, the handler runs and sets `start_reading = 1`.
- `pause()` returns because a signal was caught.
- The loop checks the flag.
- If the flag is now 1, the child breaks out and starts reading.

Why the loop is necessary

`pause()` wakes up on **any** signal, not just `SIGUSR1`.

So if the child receives something like:

- `SIGCHLD`
- `SIGPIPE`
- `SIGINT`
- or any signal from a library or thread

...it will wake up even though the parent hasn't signaled yet.

The loop ensures the child goes back to sleep until the correct signal flips the flag.

Parent side

When the parent finishes all its threads, it wakes the child:

```
c
kill(child_pid, SIGUSR1);
```

That triggers the handler, flips the flag, and the child exits the pause loop.

Important caveat

This method **can still race** if the parent sends SIGUSR1 *before* the child installs the handler and enters the loop. You must set up the handler **before** the child can possibly receive the signal.

sigwait() avoids this problem entirely, which is why it's preferred.

But if you want the pause-based version, the above is the correct pattern.

If you want, I can show you a complete parent/child program using this exact mechanism.

‘when passing an integer ID to a pthread in C, is it fine to just pass the address of my loop variable like &i’

OUTPUT: Short version: **No — passing &i from a loop variable is unsafe and will cause bugs.** Long version: here's exactly *why* and what to do instead.

Why &i is unsafe

When you do something like:

```
c
for (int i = 0; i < N; i++) {
    pthread_create(&threads[i], NULL, worker, &i);
}
```

All threads receive **the same address** — the address of the loop variable i.

Two bad things happen:

1. The loop keeps changing i

By the time the thread runs, i might already be:

- incremented
- finished the loop
- out of scope

So threads often see the **final value of i**, not the intended one.

2. Threads race to read the same memory

Multiple threads dereference the same int *, so they read whatever value happens to be there at the moment.

This leads to classic symptoms:

- All threads print the same ID
- Some threads get wrong IDs
- Some threads get garbage values
- Behavior changes depending on timing

Correct ways to pass an integer ID

Option 1: Allocate each ID separately

This is the simplest and most reliable:

```
c
for (int i = 0; i < N; i++) {
    int *id = malloc(sizeof(int));
    *id = i;
    pthread_create(&threads[i], NULL, worker, id);
}
```

Then inside the thread:

```
c
void *worker(void *arg) {
    int id = *(int *)arg;
    free(arg);
    ...
}
```

Each thread gets its own memory → no races.

Option 2: Use an array of IDs

If you don't want dynamic allocation:

```
c
int ids[N];
```

```
for (int i = 0; i < N; i++) {  
    ids[i] = i;  
    pthread_create(&threads[i], NULL, worker, &ids[i]);  
}
```

This works because each thread gets a **stable address** that never changes.

❌ Option 3: Passing &i

This is the broken version:

```
c  
  
pthread_create(&threads[i], NULL, worker, &i); // WRONG
```

Never do this.

🧠 Why this matters in your producer/pipe setup

If each producer thread needs a unique ID (for logging, debugging, or selecting data), passing &i will cause:

- producers to think they have the same ID
- unpredictable behavior
- hard-to-trace bugs in your pipe writes

Using a per-thread ID fixes all of that.

If you want, I can show you a complete producer-thread example using mutex-protected pipe writes and proper thread IDs.